

React Practice Assignment

Build a small Task Tracker app to practice core React concepts

Level: Beginner

Estimated time: 2–3 hours

Format: Vite + React

Overview

In this assignment you'll build a small **Task Tracker** app. It's intentionally simple in scope, but touches every core concept from the React basics notes: components, props, state, events, list rendering, conditional rendering, and a side effect with `useEffect`. Work through the tasks in order — each one builds on the last.

Goal: by the end, you'll have a working app where a user can add tasks, mark them complete, delete them, filter by status, and see a live count — all without reloading the page.

Setup

Create a new Vite + React project and start the dev server:

```
npm create vite@latest task-tracker -- --template react
cd task-tracker
npm install
npm run dev
```

Open the project in your editor and clear out the boilerplate in `src/App.jsx` so you have a clean starting point.

Part 1 — Static Component Structure

Before adding any interactivity, build the static shape of the app using components and props.

- 1 Create a `TaskItem` component that receives a task object as a prop and renders its title in a list item (`<li>`).
- 2 Create a `TaskList` component that receives an array of tasks as a prop and renders one `TaskItem` per task using `.map()`. Remember each item needs a unique key.
- 3 In `App.jsx`, create a hard-coded array of 3–4 sample task objects (each with an `id`, `title`, and `completed` boolean) and pass it down to `TaskList`.

Checkpoint: you should see your sample tasks rendered as a plain list on the page.

Part 2 — Adding State

Now make the task list dynamic by moving it into state.

- 1 In `App.jsx`, move your sample tasks array into a `useState` hook instead of a plain constant.
- 2 Create an `AddTaskForm` component with a text input and a submit button. It should call an `onAddTask` function (passed in as a prop) with the entered text when the form is submitted.
- 3 In `App.jsx`, write a function that adds a new task object to state (with a unique `id` and `completed: false`) and pass it down to `AddTaskForm` as `onAddTask`.
- 4 Clear the input field after a task is successfully added.

Checkpoint: typing a task title and submitting the form should add it to the list immediately, with no page reload.

Part 3 — Toggling & Deleting Tasks

- 1 Add a checkbox to `TaskItem` that reflects the task's completed value.
- 2 Pass an `onToggle` function down from `App` through `TaskList` to `TaskItem`, and call it when the checkbox changes — it should flip that task's completed value in state.
- 3 Add a delete button to `TaskItem` with an `onDelete` function (passed down the same way) that removes that task from state.
- 4 Style completed tasks differently — for example, strike-through text using a conditional `className`.

Checkpoint: checking a task should visually mark it complete, and the delete button should remove it from the list.

Part 4 — Filtering & Conditional Rendering

- 1 Add three filter buttons above the list: *All*, *Active*, and *Completed*.
- 2 Track the currently selected filter in state (e.g. a string: `"all" | "active" | "completed"`).
- 3 Before rendering, filter the tasks array based on the selected filter, and pass only the filtered tasks into `TaskList`.
- 4 If there are no tasks to show for the current filter, render a friendly message (e.g. "No tasks here yet") instead of an empty list — this is a conditional rendering check.

Part 5 — A Live Count with `useEffect`

Finally, practice a side effect by keeping the browser tab title in sync with your task count.

- 1 Add a `useEffect` that runs whenever the tasks array changes.
- 2 Inside it, count how many tasks are *not* completed, and set `document.title` to something like `"3 tasks left"`.
- 3 Double check your dependency array — the effect should only re-run when the tasks actually change.

Checkpoint: adding, completing, or deleting a task should update the text in your browser tab.

Stretch Goals (Optional)

- Persist tasks to `localStorage` so they survive a page refresh.
- Add an "edit task" feature — double-click a task to rename it.
- Show a count badge per filter button (e.g. "Active (3)").
- Extract a custom hook, e.g. `useTasks()`, that encapsulates the task state and all the add/toggle/delete logic.

Self-Check Before You're Done

- | | |
|--------------------------|---|
| ☐ | App renders without console errors or warnings |
| ☐ | Adding a task works and clears the input afterward |
| ☐ | Toggling a task's checkbox updates its completed state visually |
| ☐ | Deleting a task removes it from the list |
| ☐ | Filter buttons correctly show All / Active / Completed tasks |

☐	An empty filtered list shows a fallback message, not a blank screen
☐	Browser tab title updates automatically as tasks change
☐	Every list item has a stable, unique key prop
☐	No state is mutated directly — all updates go through setter functions

React Practice Assignment — a hands-on companion to the React Basics Notes, Cheat Sheet, and Source Code Examples.